

Compressing Long Context into Answer-Aligned Memory Embeddings for LLM Inference

Md Mostafizer Rahman¹, Md Faizul Ibne Amin², Md Shahajada Mia²,
Yutaka Watanobe², Fang Liu^{1,3*}

¹Lucy Family Institute for Data & Society, University of Notre Dame, IN, USA

²The University of Aizu, Aizuwakamatsu, Japan

³Department of Applied and Computational Mathematics and Statistics,
University of Notre Dame, IN, USA

{mrahman3, fliu2}@nd.edu, {fiamin, d8262103, yutaka}@u-aizu.ac.jp

Abstract

Large language model (LLM) inference is constrained by the quadratic scaling of self-attention and the linear scaling of the KV cache, increasing latency, energy consumption, and GPU memory demand as context length scales. Existing soft-compression methods either lack query-guided memory selection at inference time, train without answer-targeted supervision, or couple compression tightly to a specific decoder architecture. We propose a Context-to-Answer-Aligned Memory Compression (CMC) framework, which compresses long input contexts into compact Context Memory Embeddings (CMEs) aligned to any frozen decoder’s embedding space, reducing inference costs without modifying decoder weights. CMC introduces a two-tier KV cache that combines question-guided CME selection with a local context window, and trains the compressor with answer-targeted distillation from a frozen LLM. Experiments across nine encoder-decoder combinations and four QA benchmarks show that CMC consistently outperforms the baseline, achieving up to 7.3 EM and 4.0 F1 point gains on SQuAD, while reducing inference time and energy consumption by up to 20% and peak reserved GPU memory by up to 50% at 3,000 generation tokens. Ablation studies confirm that each architectural component and training objective contributes to the performance.

1 Introduction

Large language models (LLMs) have demonstrated exceptional performance across a wide range of natural language understanding and generation tasks (Brown et al., 2020; Touvron et al., 2023; Jiang et al., 2023a; Gemma Team et al., 2024). Central to their success is the Transformer architecture (Vaswani et al., 2017), whose self-attention mechanism enables rich contextual reasoning over input sequences. However, as the demand for processing longer documents, multi-turn dialogues,

and multi-document reasoning grows, a fundamental bottleneck emerges: the computational cost of self-attention scales quadratically with sequence length, and the KV cache grows linearly, consuming substantial GPU memory, increasing inference latency, and driving up energy consumption with every additional token (Pope et al., 2023; Sheng et al., 2023). These constraints become particularly acute in real-world deployments where contexts routinely span thousands of tokens, and throughput and energy efficiency are critical requirements alongside model quality.

Architectural methods address this bottleneck by redesigning the attention mechanism itself, either through sparse or local attention patterns (Beltagy et al., 2020; Zaheer et al., 2020) or by extending positional encodings to accommodate longer sequences (Chen et al., 2023; Dong et al., 2024). While effective, these approaches require substantial architectural modification or continued pre-training, making them difficult to apply to already-deployed LLMs. An alternative is hard-prompt compression, which discards tokens deemed redundant by self-information scoring or perplexity-based ranking (Li et al., 2023; Pan et al., 2024). Although model-agnostic, hard-prompt methods incur irreversible information loss and degrade on tasks requiring evidence synthesis across distant passages, such as multi-hop reasoning and long-document QA (Li et al., 2025a). In contrast, soft-compression methods encode the full context into a compact set of dense embeddings that a frozen decoder can attend to in place of the original tokens (Chevalier et al., 2023; Ge et al., 2024; Mu et al., 2023; Kim et al., 2024; Dai et al., 2025), achieving higher compression rates while preserving semantic content. Dai et al. (2025) propose PCC, a compressor-LLM framework that pretrains a lightweight encoder and converter to compress context into dense memory slots consumed by a frozen decoder. However, PCC supplies all com-

*Corresponding author

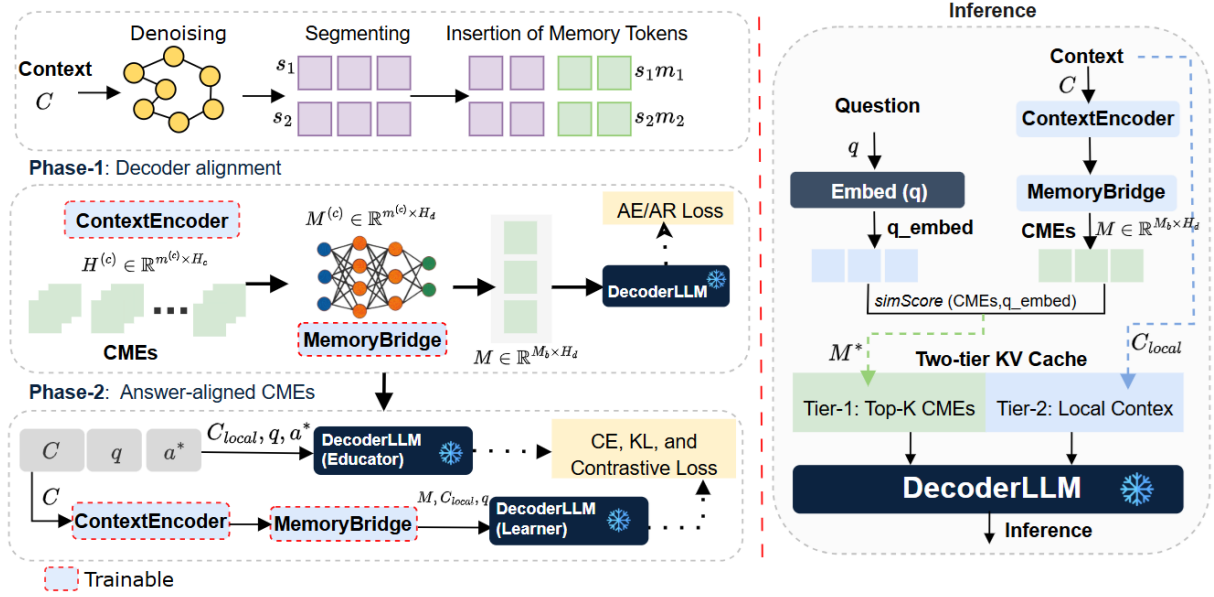


Figure 1: Overview of the CMC framework: (i) context denoising and chunked compression via ContextEncoder, (ii) cross-architecture projection via MemoryBridge, and (iii) two-tier KV cache inference via DecoderLLM with question-guided Tier-1 CME selection and Tier-2 local window.

pressed memory slots uniformly at inference time with no mechanism for query-guided selection, and trains solely on text reconstruction without answer-targeted supervision. More broadly, no existing soft-compression method simultaneously addresses cross-architecture deployment, answer-targeted training supervision, and query-guided memory selection at inference stage.

To address these gaps, we propose a **Context-to-Answer-Aligned Memory Compression (CMC)** framework, illustrated in Figure 1. CMC compresses long input contexts into compact Context Memory Embeddings (CMEs) projected into any frozen decoder’s own embedding space, enabling efficient inference without modifying decoder weights. CMC comprises three key components: (i) a trainable **ContextEncoder** that applies graph-based context denoising and chunked compression to produce CMEs via fixed placeholder tokens; (ii) a **MemoryBridge**, a normal-calibrated two-layer MLP that projects CMEs into the frozen decoder’s embedding space, enabling cross-architecture pairing of any encoder with any decoder; and (iii) a frozen **DecoderLLM** that generates responses conditioned on a two-tier KV cache combining top- K CMEs selected by cosine similarity to the input question (Tier-1) and a local context window at full token resolution (Tier-2). The ContextEncoder and MemoryBridge are trained jointly in two phases: Phase-1 on autoen-

coding and autoregressive objectives to align CMEs with the decoder, and Phase-2 on knowledge distillation and contrastive memory-answer alignment using a frozen DecoderLLM to incorporate answer-targeted supervision. The main contributions of this work are as follows.

- We propose CMC, a soft-compression framework that incorporates graph-based context denoising to improve CME quality and a two-tier KV cache inference policy that combines question-guided CME selection with a local context window, providing *both global* semantic coverage *and local* span-level precision without full-context attention.
- We propose a two-phase training strategy. Phase-1 aligns CMEs with the frozen decoder via reconstruction objectives, and Phase-2 distills answer-targeted knowledge from a frozen DecoderLLM (educator) via cross-entropy, KL divergence, and contrastive memory-answer alignment losses, introducing answer-targeted training supervision for cross-architecture soft-compression.
- We conduct comprehensive experiments across nine encoder-decoder combinations and four QA benchmarks (SQuAD, AdversarialQA, HotpotQA, CovidQA) and show that CMC consistently outperforms the baseline in EM and F1 with notable reductions in inference time, en-

energy consumption, multiply–accumulate operation (MAC), and peak GPU memory.

2 Methods

We propose the CMC framework to compress a long input context C into compact CMEs that a frozen DecoderLLM attends to in place of the original tokens. Given context C and question q , CMC learns a ContextEncoder f_{θ_c} and MemoryBridge g_ϕ such that:

$$\hat{a} = f_{\theta_d}(\cdot \mid M^*, C_{\text{local}}, q), \text{ where} \quad (1)$$

$$M^* = \text{TopK}(g_\phi(f_{\theta_c}(C)), q, K). \quad (2)$$

$\hat{a}$ is the answer generated by the DecoderLLM f_{θ_d} , M^* is the set of query-selected decoder-ready CMEs (formulated in Sections 2.2 and 2.3), and C_{local} is the local window (formulated in Section 2.5). The ContextEncoder f_{θ_c} and MemoryBridge g_ϕ are the only trainable components, whereas f_{θ_d} remains frozen throughout. The following subsections detail each component as illustrated in Figure 1.

2.1 Context Denoising, Segmentation, and Memory Tokens

We apply graph-based context denoising to remove low-salience tokens – tokens that are semantically isolated from their immediate neighbors – from the full context C before compression. Given context $C = (x_1, \dots, x_n)$, we compute the cosine similarity between each token and its g immediately downstream neighbors in embedding space. A token x_i is retained if its cosine similarity to at least one of its g subsequent neighbors meets or exceeds the tunable threshold τ :

$$C'_i = \left\{ x_i \in C \mid \max_{j=i+1}^{\min(i+g, n)} \frac{\mathbf{e}_i \cdot \mathbf{e}_j}{\|\mathbf{e}_i\| \|\mathbf{e}_j\|} \geq \tau \right\}. \quad (3)$$

x_i is discarded if C'_i is an empty set. For retained x_i , the denoised context C' of $k \in (1, g]$ tokens is divided into n_s non-overlapping segments, each of fixed length t , yielding $\mathcal{S} = \{s_1, s_2, \dots, s_{n_s}\}$, where $n_s = \lceil k/t \rceil$. To each segment s_i , we append $m^{(c)}$ fixed placeholder tokens bounded by special $\langle \text{MEM} \rangle$ and $\langle / \text{MEM} \rangle$ tokens, forming the augmented sequence $\tilde{s}_i$:

$$\tilde{s}_i = \left[s_i \parallel \langle \text{MEM} \rangle \parallel \underbrace{\mathbf{p}_1, \dots, \mathbf{p}_{m^{(c)}}}_{m^{(c)} \text{ placeholders}} \parallel \langle / \text{MEM} \rangle \right], \quad (4)$$

The number of placeholder tokens per segment is controlled by the compression rate $r = t/m^{(c)}$; a higher r yields a more compact representation at the cost of reduced context coverage. We evaluate $r \in \{2, 4, 8\}$ in our experiments.

2.2 CME Extraction with ContextEncoder

Each augmented segment $\tilde{s}_i$ in Eq. (4) is processed independently by the ContextEncoder f_{θ_c} , a trainable decoder-only causal language model. In particular, via causal self-attention, each of the $m^{(c)}$ placeholder tokens attends to all preceding chunk tokens in s_i , condensing the segment content into $m^{(c)}$ hidden state vectors $H_i^{(c)} \in \mathbb{R}^{m^{(c)} \times H_c}$ extracted as the segment-level CMEs from the final transformer layer:

$$H_i^{(c)} = f_{\theta_c}(\tilde{s}_i)[\text{placeholder positions}], \quad (5)$$

where H_c is the dimension of each hidden state vector. Though the placeholder tokens are fixed, their hidden states become informative CMEs through training the ContextEncoder weights. After processing all n_s segments, the full CME matrix is formed by concatenation:

$$\mathbf{H}^{(c)} = [H_1^{(c)}; H_2^{(c)}; \dots; H_{n_s}^{(c)}] \in \mathbb{R}^{M_b \times H_c}, \quad (6)$$

where $M_b = n_s \times m^{(c)}$ is the total memory vectors representing the full context.

2.3 Cross-Architecture Projection with MemoryBridge

The ContextEncoder and the frozen DecoderLLM may belong to different model families with different hidden dimensions H_c and H_d . MemoryBridge is a trainable two-layer MLP that projects each CME vector $\mathbf{h} \in \mathbb{R}^{H_c}$ from ContextEncoder into the DecoderLLM’s embedding space $\mathbb{R}^{H_d}$. Let $\hat{\mathbf{m}}$ denote the output of the two-layer MLP after LayerNorm¹. We apply

$$\tilde{\mathbf{m}} = \nu \tanh(\hat{\mathbf{m}}) \cdot \min\left(1, \frac{2\nu}{\|\nu \tanh(\hat{\mathbf{m}})\|_2}\right), \quad (7)$$

where $\tanh$ is applied coordinatewise and $\nu = s \cdot \bar{\nu}$, in which s is a learnable scalar and $\bar{\nu}$ denotes the average ℓ_2 norm of the frozen decoder’s input token embedding vectors, computed across all vocabulary tokens once at initialization. The first factor, $\nu \tanh(\hat{\mathbf{m}})$, bounds each coordinate but not the ℓ_2 norm of the vector; the second factor rescales

¹See Appendix A.1 for the LayerNorm formulation used.

the full vector whenever its ℓ_2 norm exceeds 2ν , and leaves it unchanged otherwise. Eq. (7) guarantees $\|\tilde{\mathbf{m}}\|_2 \leq 2\nu$, preventing the projected CMEs from exceeding twice the typical ℓ_2 norm of the DecoderLLM’s own token embeddings.

Applying MemoryBridge g_ϕ to each segment’s CME matrix $H_i^{(c)}$ yields its representation:

$$M^{(i)} = g_\phi(H_i^{(c)}) \in \mathbb{R}^{m^{(c)} \times H_d}.$$

After all n_s segments are processed, the decoder-ready representations are concatenated to

$$M = [M^{(1)}; M^{(2)}; \dots; M^{(n_s)}] \in \mathbb{R}^{M_b \times H_d}. \quad (8)$$

to form the full memory matrix M that provides the DecoderLLM with a compressed sequence of H_d -dimensional memory vectors in place of the original context tokens.

2.4 Two-Phase Training

The ContextEncoder f_{θ_c} and MemoryBridge g_ϕ are the same trainable modules optimized continuously across both phases in sequence, with the DecoderLLM f_{θ_d} frozen throughout.

Phase-1: Decoder alignment. Phase-1 aligns the CMEs with the decoder’s embedding space via autoencoding (AE) and autoregressive (AR) objectives. The AE loss conditions the decoder on M in Eq. (8) to reconstruct the original context; the AR loss conditions the decoder on the same M to reconstruct only the first half of the context, leading to the loss function in Phase 1:

$$\mathcal{L}_1 = \mathcal{L}_{\text{AE}} + \lambda_{\text{AR}} \mathcal{L}_{\text{AR}}. \quad (9)$$

This loss formulation ensures that CMEs carry sufficient information for the frozen decoder to reconstruct the context, where the AR term, with hyperparameter $\lambda_{\text{AR}} > 0$, up-weights the reconstruction of the earlier portion relative to the full-context AE objective.

Phase-2: Answer-aligned CMEs. Phase-2 fine-tunes the CMEs from Phase-1 toward answer-relevant content using the frozen DecoderLLM (educator) f_{θ_d} , which receives the local window C_{local} and question q . The same frozen weights f_{θ_d} are also invoked as the DecoderLLM (learner), conditioned on M and C_{local} . The two DecoderLLMs share identical parameters and differ only in their input.

Three objectives act jointly: a cross-entropy loss $\mathcal{L}_{\text{CE}}$ supervises the DecoderLLM (learner) on gold answer tokens; a KL divergence loss $\mathcal{L}_{\text{KL}}$ aligns the DecoderLLM (learner)’s output distribution with the DecoderLLM (educator)’s; and a contrastive loss $\mathcal{L}_{\text{CL}}$ (van den Oord et al., 2018) pulls the element-wise mean of the CMEs toward the DecoderLLM (educator)’s hidden state at the answer span. Taken together, the Phase-2 loss is:

$$\mathcal{L}_2 = \mathcal{L}_{\text{CE}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}} + \lambda_{\text{CL}} \mathcal{L}_{\text{CL}}, \quad (10)$$

where $\lambda_{\text{KL}} > 0$ and $\lambda_{\text{CL}} > 0$ are hyperparameters. Our ablation study confirms that the three components in the Phase-2 loss in Eq. (10) act jointly – removing any component degrades performance to the level of Phase-1 alone.

2.5 Inference with Two-Tier KV Cache Policy

At the inference stage, the frozen DecoderLLM generates responses conditioned on a two-tier KV cache built from the decoder-ready CME representation M and question q .

Tier-1: Question-guided top- K selection. The question tokens are embedded via the frozen decoder’s embedding layer and mean-pooled to obtain a query vector $\mathbf{q}_{\text{vec}} \in \mathbb{R}^{H_d}$. Each CME $\tilde{\mathbf{m}}_j \in M$ is scored by cosine similarity:

$$\text{score}_j = \frac{\tilde{\mathbf{m}}_j \cdot \mathbf{q}_{\text{vec}}}{\|\tilde{\mathbf{m}}_j\| \|\mathbf{q}_{\text{vec}}\|}. \quad (11)$$

The top- K CMEs are selected and re-ordered by original document position to form M^* , where K is a tunable hyperparameter controlling the breadth of semantic coverage.

Tier-2: Local context window. A W -token window C_{local} is extracted from C centered at the starting-token position α of the gold answer span, subject to context boundary constraints:

$$C_{\text{local}} = C[\text{start} : \text{start} + W], \text{ where } \quad (12) \\ \text{start} = \max(0, \min(|C| - W, \alpha - W/2)).$$

Tier-2 preserves the information critical region at full token resolution, providing the local textual precision that CMEs alone cannot supply. In our experiments, α is taken from benchmark annotations when available and otherwise computed by string matching of the information-critical region within the context C . Both CMC and the baseline use the same α in every comparison. In deployment

settings where α is unavailable, a lightweight span localization step such as CME-guided or BM25 retrieval can be used; see Appendix A.8.

KV cache assembly. The prefill input is

$$\text{input} = [M^* \parallel C_{\text{local}} \parallel q]. \quad (13)$$

During autoregressive generation, the total KV cache length is bounded by $K + W + |q|$ tokens at all generation steps regardless of document length, where $|q|$ denotes the number of tokens in the question q .

3 Experiments

3.1 Datasets

We evaluate CMC on four extractive QA benchmarks covering diverse reasoning types. SQuAD (Rajpurkar et al., 2016) is a single-hop extractive QA dataset where answers are spans within a single passage. AdversarialQA (Bartolo et al., 2020) extends this with adversarially constructed questions designed to challenge model comprehension. HotpotQA (Yang et al., 2018) requires multi-hop reasoning across multiple supporting passages, while CovidQA (Möller et al., 2020) evaluates domain-specific QA on biomedical literature. We conduct experiments under two data settings. The **controlled-data setting** uses 10,000 training and 1,000 validation samples for SQuAD, AdversarialQA, and HotpotQA, and 1,500 training and 200 validation samples for CovidQA. The **full-data setting** uses the complete training and validation splits of SQuAD, AdversarialQA, and HotpotQA to enable direct comparison with published baselines; CovidQA is evaluated under the small-data setting only due to its limited corpus size. This selection covers single-hop, adversarial, multi-hop, and domain-specific reasoning, providing a broad evaluation of CMC across varied context lengths and answer types.

3.2 Experimental Settings

All experiments are conducted on NVIDIA A100 80 GB GPU. The DecoderLLM is loaded in 4-bit NF4 quantisation with double quantisation enabled and kept frozen throughout training and inference. We evaluate three ContextEncoders – GPT2-Large, OPT-1.3B, and OPT-2.7B – paired with three frozen DecoderLLMs – Llama-3-8B-Instruct, Mistral-7B-Instruct-v0.3, and Gemma-2-9B-IT – yielding nine encoder-decoder combinations. The

ContextEncoder and MemoryBridge are trained jointly for one epoch using AdamW (learning rate $= 1 \times 10^{-5}$, $\beta = (0.9, 0.98)$). Full hyperparameter details are provided in Appendix A.1.

Evaluation Metrics. We evaluate CMC on task performance and inference efficiency. For task performance, we report Exact Match (EM) and F1. For efficiency, we measure inference time decomposed into compression, prefill, and decode stages, energy consumption in kWh via CodeCarbon (Courtney et al., 2024), MACs, and peak allocated and reserved GPU memory. Efficiency metrics are evaluated on SQuAD using GPT2-Large, Mistral, and Llama at $r = 4$, across generation token budgets $T \in \{512, 1024, 2048, 3000\}$.

3.3 Baselines

We evaluate CMC against two groups of baselines. The **primary baseline** passes the local context window directly to the frozen DecoderLLM without compression, using the gold answer offset to center the window. We run this baseline across all three DecoderLLMs using identical training data, evaluation splits, and inference settings as CMC, ensuring a controlled comparison. The second baseline type is the **published baselines**. For full-data experiments on SQuAD, HotpotQA, and AdversarialQA, we compare against one hard-prompt method – LLMingua-2 (Pan et al., 2024) – and four soft-compression methods – AutoCompressor (Chevalier et al., 2023), xRAG (Cheng et al., 2024), ICAE (Ge et al., 2024), and PCC-lite and PCC-large (Dai et al., 2025).

3.4 Results

3.4.1 Task Performance

Table 1 reports EM and F1 under both data settings at $r = 4$; full results across all nine encoder-decoder combinations and $r \in \{2, 4, 8\}$ are in Appendix A.2. **Controlled-data setting.** CMC outperforms the baselines on the majority of datasets. With Llama, CMC achieves the largest gains, improving EM by 4.6, 2.5, 5.5, and 7.0 points on SQuAD, AdversarialQA, HotpotQA, and CovidQA, respectively (average +4.9 EM). For Mistral, CMC improves on three of four datasets for a net average gain of 1.4 EM. For Gemma, CMC improves consistently across all four datasets, with the largest F1 gains on AdversarialQA (+3.1) and HotpotQA (+3.6). **Full-data setting.** CMC continues to benefit Llama and Gemma, with

DecoderLLM	Model	SQuAD		AdversarialQA		HotpotQA		CovidQA [†]		Average	
		EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
Controlled-data setting											
Llama-3 8B-Instruct	Llama (baseline)	62.4	81.0	34.3	53.3	44.1	68.1	22.5	63.8	40.8	66.6
	CMC	67.0	82.9	36.8	55.1	49.6	69.3	29.5	64.2	45.7	67.9
Mistral-7B Instruct-v0.3	Mistral (baseline)	61.0	75.9	29.6	47.9	53.0	71.1	19.0	57.6	40.7	63.1
	CMC	60.2	75.5	32.2	50.3	53.8	71.7	22.0	63.1	42.1	65.2
Gemma-2 9B-IT	Gemma (baseline)	55.1	78.1	32.9	56.4	40.1	67.0	10.5	62.1	34.7	65.9
	CMC	61.1	81.2	36.8	59.5	42.0	70.6	14.0	63.2	38.5	68.6
Full-data setting											
Llama-3 8B-Instruct	Llama (baseline)	48.28	74.27	32.47	52.55	44.31	67.28	-	-	41.69	64.70
	CMC	55.56	78.29	35.43	53.42	47.77	67.83	-	-	46.25	66.51
Mistral-7B Instruct-v0.3	Mistral (baseline)	45.36	69.26	26.37	45.91	53.41	71.13	-	-	41.71	62.10
	CMC	30.67	55.16	29.77	49.26	48.33	65.49	-	-	35.37	55.38
Gemma-2 9B-IT	Gemma (baseline)	40.91	71.68	29.77	53.17	37.80	65.60	-	-	36.16	63.48
	CMC	44.13	73.27	33.33	56.24	41.90	69.10	-	-	39.79	66.20

Table 1: EM and F1 on four QA benchmarks under controlled-data and full-data settings. **Bold** denotes the higher score per row. [†]CovidQA is evaluated under the controlled-data setting only due to its limited corpus size.

Llama gaining 7.28 EM on SQuAD (48.28→55.56) and Gemma gaining 3.22 EM (40.91→44.13). Mistral shows a mixed pattern. CMC improves over the Mistral baseline on AdversarialQA (26.37→29.77, +3.40 EM) but underperforms the Mistral baseline by 14.7 EM on SQuAD (45.4→30.7) and regresses on HotpotQA (53.4→48.3). All three decoders share an identical Phase-2 configuration, including a fixed KD step budget that does not scale with training set size, while Phase-1 steps scale directly with it ($\sim 8.8\times$ more steps on full-data vs. controlled-data SQuAD); we attribute Mistral’s SQuAD/HotpotQA regressions to this training imbalance, to which Llama and Gemma appear less sensitive under the identical settings.

Method	SQuAD		HotpotQA		Adv.QA	
	EM	F1	EM	F1	EM	F1
AutoCompressor	0.35	21.46	0.29	16.29	2.00	14.09
xRAG	3.46	18.19	16.29	27.51	3.47	13.75
ICAE	21.63	45.69	26.68	35.16	11.70	27.98
LLMLingua-2	32.18	51.20	44.18	55.72	24.80	35.41
PCC (Lite, 4 $\times$)	57.44	75.83	42.20	50.37	37.83	50.36
PCC (Large, 4 $\times$)	60.04	77.76	39.97	48.19	39.37	52.56
CMC	55.56	78.29	47.77	67.83	35.43	53.42

Table 2: Comparison of CMC with published baselines under the full-data setting on SQuAD, HotpotQA, and AdversarialQA. The best EM and F1 scores for each dataset are highlighted in bold.

Table 2 compares CMC against published baselines on the full-data setting. Although PCC-lite shares the same GPT2-Large encoder as CMC, all published baselines use different decoder architectures (Dai et al., 2025). CMC achieves the best F1 on all three datasets, surpassing PCC-Large by 0.53 F1 on SQuAD, 19.64 on HotpotQA, and 0.86 on AdversarialQA. On HotpotQA, CMC also

leads on EM (47.77 vs 39.97 for PCC-Large), a 7.8-point gain particularly notable given the multi-hop reasoning requirement. CMC underperforms PCC-Large on EM for SQuAD (55.56 vs 60.04) and AdversarialQA (35.43 vs 39.37), likely reflecting PCC’s use of a larger encoder/decoder architecture. AutoCompressor and xRAG achieve low EM on SQuAD, HotpotQA, and AdversarialQA, while ICAE and LLMLingua-2 perform substantially below CMC across all three datasets. A qualitative case study illustrating CMC behavior across four outcome categories is provided in Appendix A.9.

3.4.2 Generation and Use of CMEs

The number of CMEs generated per context chunk is controlled by the compression rate r , where a smaller r produces more CMEs per chunk and a larger r produces fewer. Figure 2 shows the EM and F1 sensitivity to $r \in \{2, 4, 8\}$ across all nine encoder-decoder combinations on SQuAD. $r = 2$ consistently yields the lowest performance. The larger CME count introduces noise into the Tier-1 prefix, diluting answer-relevant signals and making it harder for the frozen DecoderLLMs to locate the correct span. In contrast, $r = 8$ reduces the CME count to the point where important contextual information is lost, particularly for longer passages. The intermediate setting $r = 4$ achieves the best or tied-best EM in 6 of 9 combinations. However, the optimal rate is mildly decoder-dependent: for example, Llama strongly favors $r = 4$ across all encoders (EM gains of 1.1–1.7 points over $r = 8$), while Mistral and Gemma show near-flat performance between $r = 4$ and $r = 8$ (differences ≤ 0.1 EM), suggesting decoder-specific sensitivity to compression granularity.

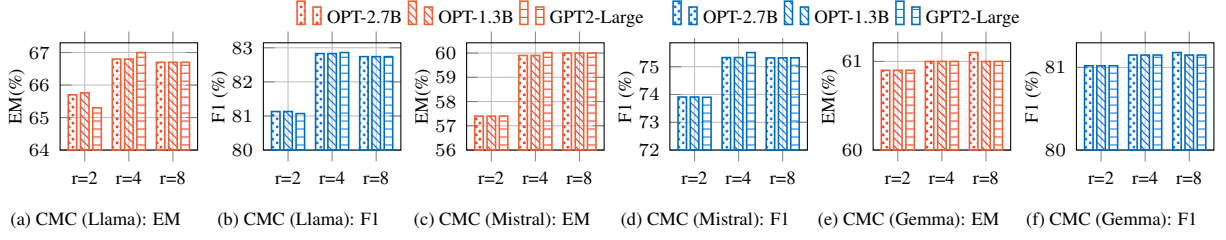


Figure 2: EM and F1 scores on SQuAD (controlled-data setting) across compression rates $r \in \{2, 4, 8\}$ for three DecoderLLMs paired with three ContextEncoders.

3.4.3 Computational Efficiency

Figure 3 shows inference time and energy cost for $T \in \{512, 1024, 2048, 3000\}$ tokens on SQuAD for three evaluation sample sizes. CMC consistently reduces both metrics relative to the baseline across all token budgets and sample sizes. At 1,000 samples, the total time reduction ranges from 4.6% at $T = 512$ (6,043 s $\rightarrow$ 5,763 s) to 20.0% at $T = 3,000$ (41,963 s $\rightarrow$ 33,562 s), with corresponding energy reductions of 4.6% and 20.3%, respectively. This scaling effect arises because CMC’s compression overhead is approximately constant (≈ 25 s regardless of T), while decode savings grow with generation length due to the bounded two-tier KV cache. The same trend holds across all sample sizes, confirming the gains reflect the two-tier KV cache architecture rather than dataset-specific effects. Stage-wise inference time and energy breakdowns for Llama and Mistral are provided in Appendix A.3.

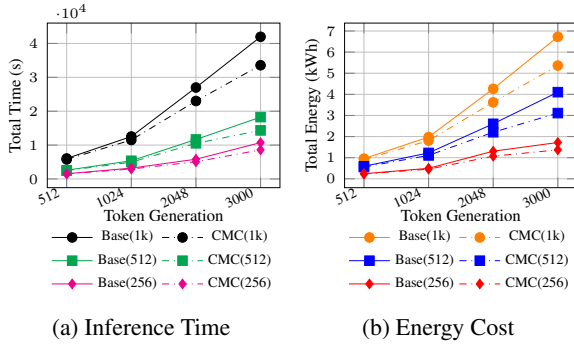


Figure 3: Inference time and energy consumption of CMC vs. the baseline on SQuAD across $T \in \{512, 1024, 2048, 3000\}$ tokens. Parenthetical labels denote sample size (256, 512, 1,000).

Figure 4 shows peak allocated and peak reserved GPU memory for Llama across $T \in \{512, 1024, 2048, 3000\}$ on SQuAD (1,000 samples, GPT2-Large, $r = 4$). Peak allocated memory remains nearly constant for CMC (≈ 8.85 GB) while the baseline increases from 10.0 GB at $T =$

512 to 10.2 GB at $T = 3,000$, yielding a reduction of ≈ 1.2 GB. Peak reserved memory reveals a more substantial difference: the baseline spikes to 19.7 GB at $T = 3,000$ while CMC maintains a near-constant 9.8 GB, corresponding to a 50% reduction. Mistral results in the appendix show an even larger effect (24.3 GB $\rightarrow$ 9.1 GB, a 62.5% reduction). These reductions arise because baseline inference accumulates KV cache entries proportionally to T , whereas CMC’s two-tier policy caps the cache at a fixed budget. Detailed memory results for both decoders are provided in Appendix A.4. Analytical attention MAC analysis in Appendix A.5 confirms that these savings are architectural rather than hardware-specific.

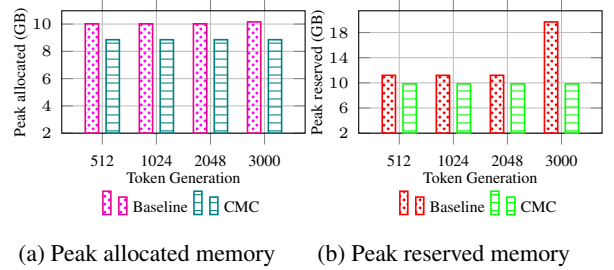


Figure 4: Peak allocated and peak reserved GPU memory (GB) for CMC vs. the baseline on 1,000 SQuAD samples across $T \in \{512, 1024, 2048, 3000\}$ tokens.

4 Ablation Study

Table 3 presents the ablation results on SQuAD. **Architectural ablations (A1–A3):** Removing graph-based denoising (A1) reduces EM by 26.6 points relative to CMC, 22.0 points below the baseline, confirming that denoising is essential for CME quality rather than merely a computational optimization. Random CME selection (A2) reduces EM by 8.1 points: the same number of CME slots reach the decoder but are decorrelated from the question, reducing Tier-1 prefix informativeness. Removing Tier-2 (A3) produces the largest drop

(−50.0 EM). The near-identical AE/AR losses between A3 and CMC (1.40/0.73 vs 1.38/0.70) confirm the deficit arises at inference rather than from degraded CME quality, establishing Tier-2 as structurally indispensable. **Phase-2 training objective ablations (A4–A6).** All converge to EM = 0.589 – identical to Phase-1 only (A6). Removing either $\mathcal{L}_{\text{KL}}$ or $\mathcal{L}_{\text{CL}}$ individually is as damaging as removing Phase-2 entirely, revealing that all three objectives must act jointly: $\mathcal{L}_{\text{CE}}$ provides token-level supervision, $\mathcal{L}_{\text{KL}}$ aligns the learner with the educator’s distribution, and $\mathcal{L}_{\text{CL}}$ grounds the CME prefix in the answer embedding space.

Model	EM	F1	ΔEM	ΔF1
Llama (baseline)	0.624	0.810	−0.046	−0.019
CMC	0.670	0.829	—	—
<i>Architecture ablations</i>				
w/o graph denoising (A1)	0.404	0.564	−0.266	−0.265
w/o query-guided top- K (A2)	0.589	0.757	−0.081	−0.072
w/o Tier-2 local window (A3)	0.170	0.247	−0.500	−0.582
<i>Training objective ablations</i>				
w/o $\mathcal{L}_{\text{CL}}$ (A4)	0.589	0.757	−0.081	−0.072
w/o $\mathcal{L}_{\text{KL}}$ (A5)	0.589	0.757	−0.081	−0.072
Phase-1 only, no Phase-2 (A6)	0.589	0.757	−0.081	−0.072

Table 3: Ablation study on SQuAD (controlled-data setting, GPT2-Large, $r = 4$, Llama). ΔEM and ΔF1 are relative to CMC.

Sensitivity analyses of local window size and denoising threshold are provided in Appendices A.6 and A.7, respectively. A **deployment analysis** of Tier-2 window localization without gold annotations is provided in Appendix A.8.

5 Related Work

Architectural methods. Sparse and local attention mechanisms (Beltagy et al., 2020; Zaheer et al., 2020) reduce quadratic complexity by restricting each token’s receptive field, while positional interpolation (Chen et al., 2023), YaRN (Peng et al., 2024), and LongLoRA (Chen et al., 2024) extend the context window of pretrained LLMs through rotary embedding modifications. FlashAttention (Dao et al., 2022) reduces memory bandwidth costs without approximation, and KV cache management strategies such as H2O (Zhang et al., 2023), and StreamingLLM (Xiao et al., 2024) evict low-importance entries during decoding. These methods require architectural modification, continued pretraining, or operate only on already-computed caches without reducing prefill cost, and none produce a compact input representation consumable by an unmodified frozen LLM.

Hard-prompt compression. Selective Context (Li et al., 2023) prunes tokens by self-information, LLM-Lingua (Jiang et al., 2023b) and LongLLM-Lingua (Jiang et al., 2024) apply perplexity-based budget control, LLM-Lingua-2 (Pan et al., 2024) distills compression decisions into a lightweight classifier, and QGC (Cao et al., 2024) weights tokens by query relevance. These methods are model-agnostic but incur irreversible information loss, as discarded tokens cannot be recovered by the decoder, making them susceptible to performance degradation on tasks requiring cross-passage evidence synthesis such as multi-hop reasoning.

Soft-compression. Soft-compression methods encode the full context into dense embeddings consumed by a frozen decoder, avoiding hard token removal. RMT (Bulatov et al., 2022) propagates memory tokens recurrently across segments. AutoCompressor (Chevalier et al., 2023) and ICAE (Ge et al., 2024) compress long documents into soft summary vectors via autoencoding objectives. Gisting (Mu et al., 2023) compresses instructions into gist tokens, and CCM (Kim et al., 2024) targets online interaction scenarios by compressing accumulating KV pairs via a conditional LoRA adapter. 500xCompressor (Li et al., 2025b) compresses the entire context into a small set of learned tokens, and then uses the KV representation of these tokens for the downstream QA task. In RAG settings, xRAG (Cheng et al., 2024) and COCOM (Rau et al., 2025) compress retrieved passages into dense representations for a single specific target decoder. Dai et al. (2025) introduce a decoupled compressor-LLM framework with cross-architecture projection but without query-guided memory selection or answer-targeted training supervision. ATACompressor (Li et al., 2025a) introduces query-guided selective encoding but does not address cross-architecture deployment or answer-targeted memory alignment. No existing method simultaneously addresses all three gaps, which CMC is designed to fill.

6 Conclusion

We presented CMC, a soft-compression framework that compresses long contexts into compact CMEs for computation- and energy- efficient inference with frozen LLMs. Across nine encoder-decoder combinations and four extractive QA benchmarks, CMC consistently outperforms the baseline in the majority of configurations under the controlled-data setting, and achieves the best F1 against pub-

lished soft-compression baselines under full-data conditions. At $T = 3,000$ tokens, CMC reduces inference time and energy by 20% and peak reserved GPU memory by up to 62.5%, enabling deployment at generation lengths that would otherwise exhaust GPU memory. Ablation studies confirm that graph-based denoising, the Tier-2 local window, and all three Phase-2 objectives are each necessary; removing any single component causes substantial performance degradation.

Limitations

While CMC demonstrates consistent improvements in EM, F1, and inference efficiency, we acknowledge the following limitations. First, all experiments are conducted on extractive QA benchmarks; the effectiveness of CMC on abstractive QA, summarisation, and RAG has not been evaluated and may require adjustments to the answer-targeted distillation objective. Moreover, CMC is trained and evaluated within each benchmark independently; cross-dataset generalization (e.g., training on SQuAD and evaluating on HotpotQA) has not been evaluated. Second, CMC uses a fixed compression rate $r \in \{2, 4, 8\}$, which may over-compress short passages or under-compress long ones; adaptive rate scheduling conditioned on context length is left for future work. Lastly, CMC is evaluated on English-language QA benchmarks only; its effectiveness on non-English or multilingual contexts, where tokenisation and semantic similarity properties differ substantially, has not been investigated.

Ethics Statement

No ethical approval was needed for this study.

Availability Statement

Code and additional resources for this study are available at <https://github.com/mostafiz26/CMC>.

References

Max Bartolo, Alastair Roberts, Johannes Welbl, Sebastian Riedel, and Pontus Stenetorp. 2020. [Beat the AI: Investigating adversarial human annotation for reading comprehension](#). *Transactions of the Association for Computational Linguistics*, 8:662–678.

Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. [Longformer: The long-document transformer](#). *arXiv preprint arXiv:2004.05150*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. [Language models are few-shot learners](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Aydar Bulatov, Yury Kuratov, and Mikhail Burtsev. 2022. [Recurrent memory transformer](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 11079–11091. Curran Associates, Inc.

Zhiwei Cao, Qian Cao, Yu Lu, Ningxin Peng, Luyang Huang, Shanbo Cheng, and Jinsong Su. 2024. [Retaining key information under high compression ratios: Query-guided compressor for LLMs](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12685–12695, Bangkok, Thailand. Association for Computational Linguistics.

Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. 2023. [Extending context window of large language models via positional interpolation](#). *arXiv preprint arXiv:2306.15595*.

Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2024. [Longlora: Efficient fine-tuning of long-context large language models](#). In *International Conference on Learning Representations*, volume 2024, pages 8220–8238.

Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. 2024. [xRAG: Extreme context compression for retrieval-augmented generation with one token](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 109487–109516. Curran Associates, Inc.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. [Adapting language models to compress contexts](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, Singapore. Association for Computational Linguistics.

Benoit Courty, Victor Schmidt, Sasha Luccioni, Goyal-Kamal, Marion Coutarel, Boris Feld, Jérémy Lecourt, Liam Connell, Amine Saboni, et al. 2024. [CodeCarbon: Estimate and track carbon emissions from machine learning computing](#).

Yuhong Dai, Jianxun Lian, Yitian Huang, Wei Zhang, Mingyang Zhou, Mingqi Wu, Xing Xie, and Hao Liao. 2025. [Pretraining context compressor for large language models with embedding-based memory](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28715–28732, Vienna, Austria. Association for Computational Linguistics.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. [Flashattention: Fast and memory-efficient exact attention with io-awareness](#).

- In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359. Curran Associates, Inc.
- Zican Dong, Junyi Li, Xin Men, Wayne Xin Zhao, Bingning Wang, Zhen Tian, Weipeng Chen, and Ji-Rong Wen. 2024. [Exploring context window of large language models via decomposed positional vectors](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 10320–10347. Curran Associates, Inc.
- Tao Ge, Hu Jing, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. [In-context autoencoder for context compression in a large language model](#). In *International Conference on Learning Representations*, volume 2024, pages 2591–2607.
- Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. [Gemma: Open models based on Gemini research and technology](#). *arXiv preprint arXiv:2403.08295*.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023a. [Mistral 7b](#). *Preprint*, arXiv:2310.06825.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023b. [LLMLingua: Compressing prompts for accelerated inference of large language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore. Association for Computational Linguistics.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. [LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 1658–1677, Bangkok, Thailand. Association for Computational Linguistics.
- Jang-Hyun Kim, Junyoung Yeom, Sangdoo Yun, and Hyun Oh Song. 2024. [Compressed context memory for online language model interaction](#). In *International Conference on Learning Representations*, volume 2024, pages 35316–35334.
- Xuancheng Li, Haitao Li, Yujia Zhou, Qingyao Ai, and Yiqun Liu. 2025a. [Atacompressor: Adaptive task-aware compression for efficient long-context processing in llms](#). In *Proceedings of the 2025 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region, SIGIR-AP 2025*, page 343–352. Association for Computing Machinery.
- Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. [Compressing context to enhance inference efficiency of large language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, Singapore. Association for Computational Linguistics.
- Zongqian Li, Yixuan Su, and Nigel Collier. 2025b. [500xCompressor: Generalized prompt compression for large language models](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25081–25091, Vienna, Austria. Association for Computational Linguistics.
- Timo M  ller, Anthony Reina, Raghavan Jayakumar, and Malte Pietsch. 2020. [COVID-QA: A question answering dataset for COVID-19](#). In *Proceedings of the 1st Workshop on NLP for COVID-19 at ACL 2020*. Association for Computational Linguistics.
- Jesse Mu, Xiang Li, and Noah Goodman. 2023. [Learning to compress prompts with gist tokens](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 19327–19352. Curran Associates, Inc.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor R  hle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. [LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression](#). In *Findings of the Association for Computational Linguistics (ACL) 2024*, pages 963–981, Bangkok, Thailand.
- Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2024. [Yarn: Efficient context window extension of large language models](#). In *International Conference on Learning Representations*, volume 2024, pages 31932–31951.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. [Efficiently scaling transformer inference](#). In *Proceedings of Machine Learning and Systems*, volume 5, pages 606–624.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. [SQuAD: 100,000+ questions for machine comprehension of text](#). In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392. Association for Computational Linguistics.
- David Rau, Shuai Wang, Herv   D  jean, St  phane Clinchant, and Jaap Kamps. 2025. [Context embeddings for efficient answer generation in retrieval-augmented generation](#). In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining, WSDM ’25*, page 493–502. Association for Computing Machinery.
- Stephen Robertson and Hugo Zaragoza. 2009. [The probabilistic relevance framework: Bm25 and beyond](#). *Found. Trends Inf. Retr.*, 3(4):333–389.

Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. 2023. [FlexGen: High-throughput generative inference of large language models with a single GPU](#). In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31094–31116. PMLR.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. [Llama 2: Open foundation and fine-tuned chat models](#). *arXiv preprint arXiv:2307.09288*.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. [Representation learning with contrastive predictive coding](#). *arXiv preprint arXiv:1807.03748*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. [Efficient streaming language models with attention sinks](#). In *International Conference on Learning Representations*, volume 2024, pages 21875–21895.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. [HotpotQA: A dataset for diverse, explainable multi-hop question answering](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. [Big bird: Transformers for longer sequences](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 17283–17297. Curran Associates, Inc.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. 2023. [H2o: Heavy-hitter oracle for efficient generative inference of large language models](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 34661–34710. Curran Associates, Inc.

A Appendix

A.1 Hyperparameters

Table 4 reports the full set of hyperparameters used across all experiments. The ContextEncoder and MemoryBridge are optimised jointly; the frozen

DecoderLLM receives no gradient updates at any stage. The two-tier KV cache hyperparameters K and W are tuned per dataset and encoder-decoder configuration within the ranges reported below.

Hyperparameter	Value
<i>Hardware</i>	
GPU	NVIDIA A100 80 GB
<i>DecoderLLM</i>	
Quantisation	4-bit NF4, double quantisation
Compute dtype	bfloat16
Decoder training	Frozen (no gradient updates)
<i>Optimiser (ContextEncoder + MemoryBridge)</i>	
Optimiser	AdamW
Learning rate	1×10^{-5}
Weight decay	0.01
β	(0.9, 0.98)
Gradient clip norm	0.6
Training epochs	1
Batch size	{4, 8} (tuned per config)
<i>ContextEncoder</i>	
Architectures	GPT2-Large, OPT-1.3B, OPT-2.7B
Max input length	3,072 tokens
<i>MemoryBridge</i>	
Architecture	Two-layer MLP
Normalisation	LayerNorm ($\epsilon = 10^{-5}$)
Activation	GELU
Output activation	Norm-calibrated tanh
<i>Graph-based token denoising</i>	
Cosine similarity threshold τ	0.05
Neighbourhood window k	3
Minimum keep interval	Every 16 tokens
IG warmup steps	200
<i>Chunking and compression</i>	
Chunk size t	256 tokens
Compression rates r	{2, 4, 8}
Memory token cap per chunk	128
Min memory tokens per chunk	4
Warmup memory tokens	8
<i>Two-tier KV cache</i>	
Tier-1 top- K CMEs	$K \in \{30, 40\}$
Tier-2 local window W	$W \in \{130, 256\}$ tokens
<i>Phase-1 AE/AR</i>	
AR loss weight λ_{AR}	1.0
<i>Phase-2 QA distillation</i>	
KD steps per epoch	800
KD batch size	2
CE loss weight λ_{CE}	1.0
KL loss weight λ_{KL}	0.5
Contrastive loss weight λ_{CL}	0.1
Contrastive temperature δ	0.07
Local window drop probability	0.5
Local window chars (educator)	1,200 – 2,000
Local window tokens (student)	130
<i>Inference</i>	
Max new tokens	32
Decoder max length	1,536 tokens

Table 4: Hyperparameters used in all CMC experiments.

LayerNorm details. All LayerNorm operations in MemoryBridge use PyTorch’s default `nn.LayerNorm`: $\text{LayerNorm}(\mathbf{x}) = \gamma \odot (\mathbf{x} - \mu) / \sqrt{\sigma^2 + \epsilon + \beta}$, where μ and σ^2 are the mean and variance of $\mathbf{x}$ over the feature dimension, $\epsilon = 10^{-5}$ (Table 4), and γ, β are learnable per-dimension scale and shift parameters trained jointly with the rest of MemoryBridge.

A.2 Detailed Results per Dataset

Tables 5–8 report EM and F1 for all nine encoder-decoder combinations across $r \in \{2, 4, 8\}$ under the controlled-data setting. **Bold** denotes the best CMC result per DecoderLLM group. Three findings across these tables support the design choices made in CMC. First, OPT-1.3B and OPT-2.7B produce near-identical results across all datasets and decoders, while GPT2-Large — the smallest encoder — matches or outperforms both, confirming that ContextEncoder scale does not drive performance. Second, Gemma shows near-zero variation across all encoder and compression rate combinations (≤ 0.001 EM), indicating that the two-tier KV cache policy is effective regardless of decoder sensitivity to compression granularity. Third, CovidQA shows consistent improvements across all nine configurations despite only 1,500 training samples, demonstrating that the two-phase training strategy generalises to low-resource domain-specific settings.

Additionally, Table 9 reports average EM and F1 across all encoder-decoder combinations and datasets. The overall average confirms $r = 4$ as the best compression rate (EM = 0.4157, F1 = 0.6678), marginally outperforming $r = 8$ (EM = 0.4152) and more substantially outperforming $r = 2$ (EM = 0.4107).

A.3 Inference Efficiency

Tables 10–13 report stage-wise inference time and energy for Llama and Mistral on SQuAD across three sample sizes and $T \in \{512, 1024, 2048, 3000\}$. These tables extend the 1,000-sample Llama results in Section 3.4.3 to smaller evaluation sets and Mistral. We present two additional observations. First, the compression overhead scales linearly with sample size — approximately 6 s at 256 samples, 13 s at 512, and 26 s at 1,000 — and is independent of T , confirming that CMC’s compression cost is determined by the number of contexts processed rather than the generation length. Second, CMC

prefill time is marginally lower than the baseline across all conditions, reflecting the shorter local context window relative to the uncompressed prompt. Moreover, Mistral shows consistent savings matching Llama ($\approx 20\%$ at $T = 3,000$, 1,000 samples), confirming that the efficiency gains are decoder-independent.

A.4 Peak GPU Memory

Tables 14 and 15 report peak allocated and peak reserved GPU memory for Llama and Mistral across three sample sizes and $T \in \{512, 1024, 2048, 3000\}$, extending the 1,000-sample Llama results in Section 3.4.3. Peak allocated memory is stable across T for CMC (≈ 8.85 GB for Llama, ≈ 8.43 GB for Mistral) regardless of sample size. The baseline is also stable for Llama (≈ 10.0 GB) but grows with T for Mistral (9.1 GB at $T = 512$ to 11.1 GB at $T = 3,000$), reflecting Mistral’s longer tokenised prompts and larger KV cache growth per step. The key finding is in peak reserved memory. CMC maintains a near-constant footprint across all sample sizes and token budgets (≈ 9.8 GB for Llama, ≈ 9.1 GB for Mistral), while the baseline spikes sharply at large T : up to 19.7 GB for Llama and 24.3 GB for Mistral at $T = 3,000$. This confirms that the bounded two-tier KV cache prevents memory growth regardless of decoder architecture, sample size, or generation length.

A.5 Attention MACs Analysis

Attention MACs are estimated analytically as $\text{MACs}_t = 2 \cdot n_h \cdot d_h \cdot n_l \cdot L_t$ per generation step, where $L_t \leq K + W + |q|$ for CMC vs. $L_t = P + t$ for the baseline, with P denoting the combined length of the baseline’s context window and question, and t the number of tokens generated so far. Absolute values differ across decoders due to architecture ($n_l = 42$ for Gemma vs. 32 for Llama and Mistral) and tokeniser-dependent prompt lengths. Table 16 shows that the $K+W$ bound reduces attention MACs by 27.0% on average (6.6%–51.3%) across all nine combinations. Reductions are largest on SQuAD (34.0% average) where longer prompt lengths make the fixed cache bound more effective. Smaller reductions on HotpotQA for Llama and Gemma (6.6% and 8.3%) reflect shorter baseline prompts in that split. These reductions are consistent with the empirical efficiency gains in Section 3.4.3, confirming that CMC’s efficiency advantage is architectural rather

DecoderLLM	Model	ContextEncoder	r	EM	F1
Llama-3 8B-Instruct	Llama (baseline)	—	—	0.6240	0.8097
	CMC	OPT-2.7B	2/4/8	0.6570/0.6680/0.6670	0.8113/0.8283/0.8274
		OPT-1.3B	2/4/8	0.6570/0.6680/0.6670	0.8113/0.8283/0.8274
		GPT2-Large	2/4/8	0.6530/ 0.6700 /0.6670	0.8107/ 0.8286 /0.8274
Mistral-7B Instruct-v0.3	Mistral (baseline)	—	—	0.6100	0.7586
	CMC	OPT-2.7B	2/4/8	0.5740/0.5990/0.6000	0.7391/0.7533/0.7532
		OPT-1.3B	2/4/8	0.5740/0.5990/0.6000	0.7391/0.7533/0.7532
		GPT2-Large	2/4/8	0.5740/ 0.6020 /0.6000	0.7390/ 0.7551 /0.7532
Gemma-2 9B-IT	Gemma (baseline)	—	—	0.5510	0.7805
	CMC	OPT-2.7B	2/4/8	0.6090/0.6100/ 0.6110	0.8102/0.8115/ 0.8118
		OPT-1.3B	2/4/8	0.6090/0.6100/0.6100	0.8102/0.8115/0.8115
		GPT2-Large	2/4/8	0.6090/0.6100/0.6100	0.8102/0.8115/0.8115

Table 5: EM and F1 results on **SQuAD** (controlled-data setting).

DecoderLLM	Model	ContextEncoder	r	EM	F1
Llama-3 8B-Instruct	Llama (baseline)	—	—	0.3430	0.5329
	CMC	OPT-2.7B	2/4/8	0.3740/0.3660/0.3670	0.5328/0.5501/0.5495
		OPT-1.3B	2/4/8	0.3740/0.3660/0.3670	0.5328/0.5501/0.5495
		GPT2-Large	2/4/8	0.3750 /0.3680/0.3650	0.5330/ 0.5509 /0.5464
Mistral-7B Instruct-v0.3	Mistral (baseline)	—	—	0.2960	0.4786
	CMC	OPT-2.7B	2/4/8	0.3050/0.3200/0.3200	0.4888/0.5013/0.5012
		OPT-1.3B	2/4/8	0.3050/0.3200/0.3200	0.4888/0.5013/0.5012
		GPT2-Large	2/4/8	0.3060/ 0.3220 /0.3180	0.4913/ 0.5029 /0.5008
Gemma-2 9B-IT	Gemma (baseline)	—	—	0.3290	0.5641
	CMC	OPT-2.7B	2/4/8	0.3660/0.3680/0.3680	0.5921/0.5952/0.5952
		OPT-1.3B	2/4/8	0.3660/0.3680/0.3680	0.5921/0.5952/0.5952
		GPT2-Large	2/4/8	0.3650/ 0.3680 /0.3680	0.5916/ 0.5952 /0.5952

Table 6: EM and F1 results on **AdversarialQA** (controlled-data setting).

than hardware-specific.

A.6 Local Window Size Sensitivity

Figure 5 shows EM and F1 as a function of the Tier-2 local window size W on SQuAD. EM increases monotonically from 0.140 at $W = 50$ to 0.354 at $W = 200$, then plateaus at $W = 256$ (EM = 0.355), confirming that larger local windows consistently benefit extractive QA by providing more surrounding context for precise span extraction. The plateau at $W \geq 200$ suggests diminishing returns once the window captures the full answer region for most passages. The default $W = 130$ achieves EM = 0.304, which is 5.0 points below the $W = 200$ (EM = 0.354), indicating that $W = 200$ is a stronger default for deployment when the additional KV budget ($K + W = 230$ tokens) is available. The finding is consistent with the 200-sample evaluation, which shows the same monotonic pattern and plateau boundary, confirming robustness to sample size.

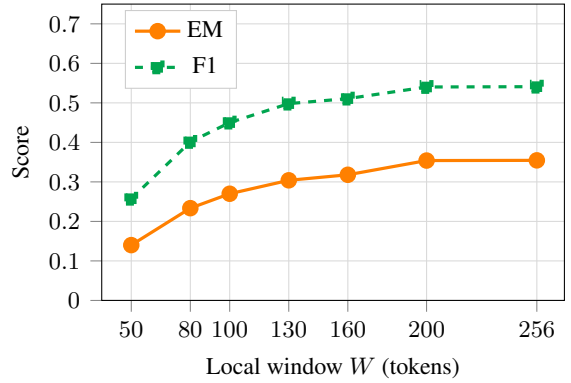


Figure 5: Local window W sensitivity on SQuAD (full-data setting, GPT2-Large, $r = 4$, Llama, $K = 30$, 2000 validation samples). EM and F1 increase monotonically with W , plateauing at $W \geq 200$.

A.7 Denoising Threshold (τ) Sensitivity

Figure 6 shows EM and F1 as a function of the graph-based context denoising threshold τ on SQuAD. The results reveal a binary pattern. At

DecoderLLM	Model	ContextEncoder	r	EM	F1
Llama-3 8B-Instruct	Llama (baseline)	—	—	0.4410	0.6814
	CMC	OPT-2.7B	2/4/8	0.4610/0.4650/0.4610	0.6554/0.6634/0.6619
		OPT-1.3B	2/4/8	0.4880/0.4810/0.4790	0.6907/0.6933/0.6922
		GPT2-Large	2/4/8	0.4960 /0.4840/0.4610	0.6934/ 0.6943 /0.6619
Mistral-7B Instruct-v0.3	Mistral (baseline)	—	—	0.5300	0.7106
	CMC	OPT-2.7B	2/4/8	0.4810/0.4910/0.4930	0.6540/0.6668/0.6672
		OPT-1.3B	2/4/8	0.5260/0.4910/ 0.5380	0.7042/0.6668/ 0.7169
		GPT2-Large	2/4/8	0.4810/0.5360/ 0.5380	0.6544/0.7158/ 0.7169
Gemma-2 9B-IT	Gemma (baseline)	—	—	0.4010	0.6700
	CMC	OPT-2.7B	2/4/8	0.4200/0.4200/0.4150	0.7059/0.7059/0.7020
		OPT-1.3B	2/4/8	0.4200/ 0.4200 /0.4150	0.7059/ 0.7059 /0.7020
		GPT2-Large	2/4/8	0.4200/0.4200/0.4150	0.7059/0.6949/0.6939

Table 7: EM and F1 results on **HotpotQA** (controlled-data setting).

DecoderLLM	Model	ContextEncoder	r	EM	F1
Llama-3 8B-Instruct	Llama (baseline)	—	—	0.2250	0.6383
	CMC	OPT-2.7B	2/4/8	0.2950/ 0.2950 /0.2950	0.6416/ 0.6416 /0.6416
		OPT-1.3B	2/4/8	0.2900/0.2900/0.2900	0.6412/0.6392/0.6392
		GPT2-Large	2/4/8	0.2900/0.2900/0.2900	0.6412/0.6392/0.6412
Mistral-7B Instruct-v0.3	Mistral (baseline)	—	—	0.1900	0.5761
	CMC	OPT-2.7B	2/4/8	0.2150/0.2200/0.2150	0.6238/0.6305/0.6238
		OPT-1.3B	2/4/8	0.2150/0.2200/0.2150	0.6238/0.6305/0.6238
		GPT2-Large	2/4/8	0.2150/ 0.2200 /0.2150	0.6238/ 0.6305 /0.6238
Gemma-2 9B-IT	Gemma (baseline)	—	—	0.1050	0.6209
	CMC	OPT-2.7B	2/4/8	0.14/0.14/0.14	0.6323/0.6323/0.6323
		OPT-1.3B	2/4/8	0.14/0.14/0.14	0.6323/0.6323/0.6323
		GPT2-Large	2/4/8	0.14/0.14/0.14	0.6323/0.6323/0.6323

Table 8: EM and F1 results on **CovidQA**.

$\tau = 0$ (no denoising), EM drops to 0.307 — a 19.9-point degradation relative to any denoising setting ($\tau \geq 0.02$, EM = 0.507). This confirms that graph-based context denoising is critical for CME quality: without it, the ContextEncoder compresses all tokens including highly redundant ones, producing a noisier and less informative CME prefix. For $\tau \geq 0.02$, EM and F1 are identical across all tested values (EM = 0.507, F1 = 0.739), with token retention stabilising at 14.0%. This plateau arises because the keep_min_every = 16 constraint — which forces retention of at least one token every 16 positions — becomes the binding constraint for $\tau \geq 0.02$. Once this floor dominates, the exact value of τ has no further effect on which tokens are retained. The default $\tau = 0.05$ is therefore well within the stable region and robust to any choice in the range [0.02, 0.20].

A.8 Tier-2 Window Localization Analysis

In main experiments, the Tier-2 local window is centered using the gold answer character offset α provided by benchmark annotations, applied identically to both CMC and the baseline so that α is a controlled variable rather than information available only to CMC. While this is standard practice in extractive QA evaluation, α is unavailable at deployment time. Table 17 quantifies the performance gap between the oracle offset and three deployment-realistic alternatives on 2,000 validation samples from SQuAD and AdversarialQA (full-data setting) and HotpotQA (controlled-data setting) (GPT2-Large, $r = 4$, Llama, $K = 30$, $W = 140$). **BM25 top-1 sentence** (Robertson and Zaragoza, 2009) ranks context sentences by relevance to the question and uses the midpoint of the top-ranked sentence as α . **CME-guided localization** uses the character offset of the chunk that produced the highest-norm CME as α , requiring no external library. **Midpoint heuristic** always cen-

DecoderLLM	ContextEncoder	$r = 2$		$r = 4$		$r = 8$	
		EM	F1	EM	F1	EM	F1
Llama-3 8B-Instruct	OPT-2.7B	0.4468	0.6603	0.4485	0.6709	0.4475	0.6701
	OPT-1.3B	0.4522	0.6690	0.4512	0.6777	0.4507	0.6771
	GPT2-Large	0.4535	0.6696	0.4530	0.6783	0.4457	0.6692
Mistral-7B Instruct-v0.3	OPT-2.7B	0.3937	0.6264	0.4075	0.6380	0.4070	0.6363
	OPT-1.3B	0.4050	0.6390	0.4075	0.6380	0.4183	0.6488
	GPT2-Large	0.3940	0.6271	0.4200	0.6511	0.4178	0.6487
Gemma-2 9B-IT	OPT-2.7B	0.3837	0.6851	0.3845	0.6862	0.3835	0.6853
	OPT-1.3B	0.3837	0.6851	0.3845	0.6862	0.3832	0.6852
	GPT2-Large	0.3835	0.6850	0.3845	0.6835	0.3832	0.6832
Overall average		0.4107	0.6607	0.4157	0.6678	0.4152	0.6671

Table 9: Average EM and F1 across four datasets (SQuAD, AdversarialQA, HotpotQA, CovidQA) under controlled-data setting, grouped by DecoderLLM, ContextEncoder, and compression rate $r \in \{2, 4, 8\}$. **Bold** denotes the best average per DecoderLLM–ContextEncoder pair.

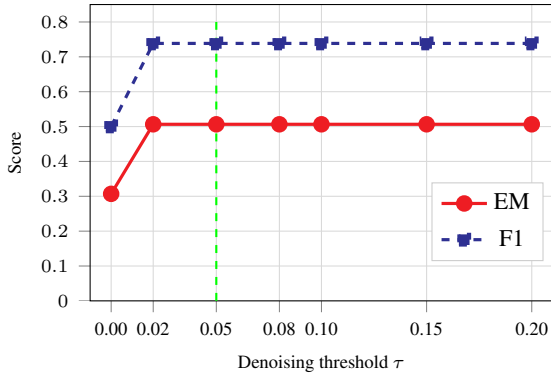


Figure 6: Denoising threshold τ sensitivity on SQuAD (full-data setting, GPT2-Large, $r = 4$, Llama, $K = 30$, $W = 140$, 2,000 validation samples). No denoising ($\tau = 0$) reduces EM by 19.9 points. For $\tau \geq 0.02$, EM and F1 are identical as the keep_min_every= 16 floor becomes the binding constraint. The default $\tau = 0.05$ (dashed line) lies within the stable region.

ties at $\alpha = |C|/2$, using no information about the question or answer at all.

On SQuAD, oracle-free localization stays close to the oracle: BM25 costs only 0.4 EM points ($\Delta\text{EM} = -0.004$, 99% retention), CME-guided costs 2.0 points (-0.020 , 95%), and even the midpoint heuristic retains 91% of oracle EM. HotpotQA shows the same pattern, and more strongly: BM25, CME-guided, and the midpoint heuristic all retain 97–98% of oracle EM, with CME-guided (0.415) and the midpoint heuristic (0.414) essentially matching BM25 (0.412). We attribute this robustness to HotpotQA’s multi-hop structure, where answer-supporting evidence is distributed across multiple passages rather than concentrated at a single span. AdversarialQA shows the largest oracle dependence (-0.052 to -0.069 , 78–84% reten-

tion), consistent with its adversarially constructed questions being designed to resist the shallow lexical and positional cues that BM25 and the midpoint heuristic rely on. These experimental results show that CMC’s Tier-2 benefit comes from the presence of a local context window, not from privileged access to the gold answer location: oracle-free localization retains the large majority of oracle performance (78–99%) across single-hop, adversarial, and multi-hop QA datasets.

A.9 Case Study

Table 18 presents four representative examples from the SQuAD validation set illustrating the behaviour of CMC relative to the baseline across four outcome categories. **Case 1** shows a straightforward factual question where both CMC and the baseline correctly extract the gold answer (*oxides*), confirming that CMC preserves answer quality on simple single-token spans. **Case 2** demonstrates a clear CMC advantage. The context contains two monetary figures — £43 million (the original gallery budget) and £76 million (the revised cost estimate). The baseline anchors to the more prominent earlier mention and returns the incorrect figure. The CME prefix supplies sufficient global context for CMC to distinguish between the two figures and return the correct answer. **Case 3** illustrates a limitation of CMC. Both CMC and the baseline locate the correct passage, but CMC produces *1979* rather than the gold *June 1979*, losing the month through compression. This suggests that fine-grained temporal spans are susceptible to information loss when the relevant token is not well-represented in the CME prefix. **Case 4** shows a partial improvement. The gold answer is a full

Samples	Model	T	Com. (s)	Pre. (s)	Dec. (s)	Total (s)
256	Baseline	512	-	7.61	1,535.77	1,543.38
	CMC		6.29	6.79	1,464.63	1,477.71
	Baseline	1024	-	7.71	3,194.18	3,201.91
	CMC		6.21	6.87	2,930.89	2,943.98
	Baseline	2048	-	7.40	5,815.22	5,822.62
	CMC		6.60	6.31	5,067.51	5,080.43
	Baseline	3000	-	7.61	10,717.09	10,724.70
	CMC		6.50	6.89	8,576.12	8,589.51
512	Baseline	512	-	16.60	2,574.88	2,591.49
	CMC		12.78	16.59	2,445.59	2,474.97
	Baseline	1024	-	17.51	5,353.99	5,371.51
	CMC		12.96	12.61	4,931.13	4,956.70
	Baseline	2048	-	14.88	11,639.93	11,654.82
	CMC		13.49	12.74	10,496.74	10,522.99
	Baseline	3000	-	17.67	18,233.12	18,250.79
	CMC		12.82	15.07	14,312.29	14,340.18
1000	Baseline	512	-	31.57	6,011.68	6,043.28
	CMC		25.14	26.94	5,711.32	5,763.45
	Baseline	1024	-	31.46	12,498.05	12,529.52
	CMC		25.26	27.11	11,451.62	11,504.01
	Baseline	2048	-	31.62	26,954.16	26,985.80
	CMC		25.51	27.75	22,958.75	23,012.03
	Baseline	3000	-	31.37	41,931.82	41,963.20
	CMC		25.69	27.13	33,508.74	33,561.58

Table 10: Stage-wise inference time (seconds) for **Llama-3-8B-Instruct** on SQuAD across three sample sizes (GPT2-Large ContextEncoder, $r = 4$, batch size 8). Com. = compression; Pre. = prefill; Dec. = decode. Baseline compression time is zero.

sentence (*The city’s residents fled to the north*), which neither model matches exactly. The baseline retrieves only *the north* ($F1 = 0.40$), while CMC retrieves *to the north* ($F1 = 0.55$), a longer span that better overlaps with the gold answer. This illustrates that CMC can improve partial answer coverage even when an exact match is not achieved.

Samples	Model	T	Com. (kWh)	Pre. (kWh)	Dec. (kWh)	Total (kWh)
256	Baseline	512	-	0.0012	0.2392	0.2404
	CMC		0.0010	0.0011	0.2278	0.2298
	Baseline	1024	-	0.0012	0.4966	0.4978
	CMC		0.0010	0.0011	0.4555	0.4576
	Baseline	2048	-	0.0017	1.3063	1.3079
	CMC		0.0014	0.0013	1.0681	1.0709
	Baseline	3000	-	0.0012	1.7187	1.7200
	CMC		0.0010	0.0011	1.3696	1.3718
512	Baseline	512	-	0.0038	0.5900	0.5938
	CMC		0.0029	0.0037	0.5496	0.5562
	Baseline	1024	-	0.0040	1.2227	1.2267
	CMC		0.0029	0.0028	1.1025	1.1082
	Baseline	2048	-	0.0033	2.6022	2.6056
	CMC		0.0028	0.0027	2.2001	2.2056
	Baseline	3000	-	0.0040	4.0996	4.1036
	CMC		0.0028	0.0033	3.1068	3.1128
1000	Baseline	512	-	0.0050	0.9514	0.9564
	CMC		0.0040	0.0043	0.9043	0.9125
	Baseline	1024	-	0.0050	1.9723	1.9773
	CMC		0.0040	0.0043	1.8009	1.8092
	Baseline	2048	-	0.0050	4.2575	4.2625
	CMC		0.0040	0.0044	3.6124	3.6208
	Baseline	3000	-	0.0050	6.7204	6.7254
	CMC		0.0041	0.0043	5.3538	5.3623

Table 11: Stage-wise energy consumption (kWh) for **Llama-3-8B-Instruct** on SQuAD across three sample sizes (GPT2-Large ContextEncoder, $r = 4$, batch size 8). Stage-wise energy is apportioned proportionally to wall-clock time.

Samples	Model	T	Com. (s)	Pre. (s)	Dec. (s)	Total (s)
256	Baseline	512	-	8.64	1,537.28	1,545.92
	CMC		6.42	7.27	1,461.70	1,475.39
	Baseline	1024	-	8.44	3,190.70	3,199.14
	CMC		6.33	7.03	2,919.23	2,932.59
	Baseline	2048	-	8.47	6,864.73	6,873.19
	CMC		6.41	6.94	5,846.33	5,859.69
	Baseline	3000	-	8.86	9,094.45	9,103.32
	CMC		6.42	6.21	7,429.81	7,442.45
512	Baseline	512	-	16.73	3,061.61	3,078.34
	CMC		12.08	14.14	2,911.43	2,937.66
	Baseline	1024	-	16.75	5,415.73	5,432.49
	CMC		12.94	12.42	5,179.11	5,204.47
	Baseline	2048	-	15.98	11,465.26	11,481.24
	CMC		13.03	12.34	10,250.06	10,275.45
	Baseline	3000	-	15.54	18,180.74	18,196.29
	CMC		12.91	13.25	14,883.97	14,910.14
1000	Baseline	512	-	33.08	5,179.30	5,212.39
	CMC		26.00	28.63	5,058.98	5,113.63
	Baseline	1024	-	31.34	10,657.52	10,688.87
	CMC		27.17	24.45	10,295.92	10,347.56
	Baseline	2048	-	34.24	26,854.94	26,889.19
	CMC		24.87	28.05	22,832.35	22,885.29
	Baseline	3000	-	34.29	41,807.04	41,841.34
	CMC		26.09	27.79	33,354.42	33,408.33

Table 12: Stage-wise inference time (seconds) for **Mistral-7B-Instruct-v0.3** on SQuAD across three sample sizes (GPT2-Large ContextEncoder, $r = 4$, batch size 8). Com. = compression; Pre. = prefill; Dec. = decode.

Samples	Model	T	Com. (kWh)	Pre. (kWh)	Dec. (kWh)	Total (kWh)
256	Baseline	512	-	0.0014	0.2464	0.2478
	CMC		0.0010	0.0012	0.2335	0.2357
	Baseline	1024	-	0.0014	0.5199	0.5213
	CMC		0.0010	0.0011	0.4727	0.4749
	Baseline	2048	-	0.0014	1.1076	1.1089
	CMC		0.0010	0.0011	0.9351	0.9373
	Baseline	3000	-	0.0020	2.0874	2.0895
	CMC		0.0013	0.0013	1.6039	1.6066
512	Baseline	512	-	0.0027	0.4966	0.4993
	CMC		0.0020	0.0023	0.4713	0.4756
	Baseline	1024	-	0.0037	1.1919	1.1956
	CMC		0.0028	0.0027	1.1356	1.1411
	Baseline	2048	-	0.0037	2.6172	2.6209
	CMC		0.0027	0.0026	2.1315	2.1367
	Baseline	3000	-	0.0036	4.1707	4.1743
	CMC		0.0028	0.0029	3.2180	3.2237
1000	Baseline	512	-	0.0072	1.1238	1.1310
	CMC		0.0054	0.0060	1.0550	1.0664
	Baseline	1024	-	0.0068	2.3237	2.3306
	CMC		0.0055	0.0050	2.1009	2.1115
	Baseline	2048	-	0.0055	4.3293	4.3349
	CMC		0.0040	0.0045	3.6503	3.6587
	Baseline	3000	-	0.0056	6.7946	6.8001
	CMC		0.0042	0.0044	5.3397	5.3483

Table 13: Stage-wise energy consumption (kWh) for **Mistral-7B-Instruct-v0.3** on SQuAD across three sample sizes (GPT2-Large ContextEncoder, $r = 4$, batch size 8). Stage-wise energy is apportioned proportionally to wall-clock time.

Samples	Model	T	Peak Alloc. (GB)		Peak Res. (GB)	
			Base	CMC	Base	CMC
256	Baseline vs CMC	512	10.033	8.916	11.234	9.742
		1024	9.968	8.848	11.125	9.742
		2048	9.970	8.806	11.100	9.717
		3000	10.151	8.848	15.332	9.742
512	Baseline vs CMC	512	10.018	8.850	11.172	9.807
		1024	10.027	8.859	12.436	11.381
		2048	10.018	8.850	11.172	9.807
		3000	10.160	8.850	18.617	9.807
1000	Baseline vs CMC	512	10.017	8.853	11.197	9.836
		1024	10.017	8.853	11.197	9.836
		2048	10.017	8.853	11.199	9.838
		3000	10.159	8.853	19.689	9.836

Table 14: Peak allocated and peak reserved GPU memory (GB) for **Llama-3-8B-Instruct** on SQuAD (GPT2-Large, $r = 4$, batch size 8). Peak allocated memory is measured via `torch.cuda.max_memory_allocated()` and peak reserved memory via `torch.cuda.max_memory_reserved()`.

Samples	Model	T	Peak Alloc. (GB)		Peak Res. (GB)	
			Base	CMC	Base	CMC
256	Baseline vs CMC	512	9.052	8.432	10.197	9.084
		1024	9.266	8.434	10.346	9.084
		2048	10.203	8.434	14.162	9.084
		3000	11.075	8.397	24.262	9.082
512	Baseline vs CMC	512	9.073	8.449	10.434	9.146
		1024	9.276	8.431	10.328	9.082
		2048	10.192	8.431	14.125	9.082
		3000	11.085	8.431	24.262	9.082
1000	Baseline vs CMC	512	9.074	8.431	10.221	9.082
		1024	9.276	8.431	10.328	9.082
		2048	10.213	8.449	14.166	9.109
		3000	11.084	8.449	24.303	9.109

Table 15: Peak allocated and peak reserved GPU memory (GB) for **Mistral-7B-Instruct-v0.3** on SQuAD (GPT2-Large, $r = 4$, batch size 8). Peak reserved memory is measured via `torch.cuda.max_memory_reserved()`.

Dataset	Decoder	Baseline	CMC	Reduction
SQuAD	Llama	1.61	1.12	30.6%
	Mistral	1.82	1.12	38.1%
	Gemma	3.00	2.01	33.0%
AdversarialQA	Llama	0.71	0.55	22.0%
	Mistral	0.77	0.56	27.4%
	Gemma	2.58	1.94	24.9%
HotpotQA	Llama	0.60	0.56	6.6%
	Mistral	0.72	0.35	51.3%
	Gemma	1.56	1.43	8.3%

Table 16: Estimated attention MACs (Billion) for CMC vs. the baseline ($r = 4$, $W = 130$, $K = 40$).

Dataset	Tier-2 localisation	EM	F1	Δ EM
SQuAD	Oracle (α from annotations)	0.408	0.591	—
	BM25 top-1 sentence	0.404	0.585	−0.004
	CME-guided (top-1 CME chunk)	0.388	0.556	−0.020
	Midpoint heuristic	0.373	0.533	−0.035
AdversarialQA	Oracle	0.321	0.501	—
	BM25	0.269	0.428	−0.052
	CME-guided	0.264	0.405	−0.057
	Midpoint heuristic	0.252	0.389	−0.069
HotpotQA	Oracle	0.423	0.634	—
	BM25	0.412	0.621	−0.011
	CME-guided	0.415	0.622	−0.008
	Midpoint heuristic	0.414	0.624	−0.009

Table 17: Effect of Tier-2 window localization method on SQuAD (full-data), AdversarialQA (full-data), and HotpotQA (controlled-data) (GPT2-Large, $r = 4$, Llama, $K = 30$, $W = 140$, 2,000 validation samples per dataset). Δ EM is relative to the oracle within each dataset.

Case 1	<i>Both Win (Baseline✓ / CMC✓)</i>
Question: What is the usual form of oxygen bound compounds?	
Gold answer: oxides	
Baseline: oxides	EM = 1 / F1 = 1.00
CMC: oxides	EM = 1 / F1 = 1.00
<i>Context:</i> Due to its electronegativity, oxygen forms chemical bonds with almost all other elements to give corresponding oxides. The surface of most metals, such as aluminium and titanium, are oxidized in the presence of air and become coated with a thin film of oxide that passivates the metal and slows further corrosion. Many oxides of the transition metals are non-stoichiometric compounds, with slightly less metal than the chemical formula would show. For example, the mineral FeO (wüstite) is written as Fe 1 - xO, where x is usually around 0.05.	
Case 2	<i>CMC wins (Baseline✗ / CMC✓)</i>
Question: What is the estimated cost of the V&A branded gallery?	
Gold answer: £76 million	
Baseline: £43 million	EM = 0 / F1 = 0.50
CMC: £76 million	EM = 1 / F1 = 1.00
<i>Context:</i> The V&A is in discussion with the University of Dundee, University of Abertay, Dundee City Council and the Scottish Government with a view to opening a new £43 million gallery in Dundee that would use the V&A brand although it would be funded through and operated independently. As of 2015, with costs estimated at £76 million, it is the most expensive gallery project ever undertaken in Scotland. The V&A Dundee will be on the city's waterfront and is intended to focus on fashion, architecture, product design, graphic arts and photography. It is planned that it could open within five years. Dundee City Council is expected to pay a major part of the running costs. The V&A is not contributing financially, but will be providing expertise, loans and exhibitions.	
Case 3	<i>Baseline wins (Baseline✓ / CMC✗)</i>
Question: When was 7 Lincoln Square completed?	
Gold answer: June 1979	
Baseline: June 1979	EM = 1 / F1 = 1.00
CMC: 1979	EM = 0 / F1 = 0.67
<i>Context:</i> Meanwhile, ABC News, which formed as a newly separate division, sought to become a global leader in television news. In 1977, Roone Arledge was named president of the new ABC News in addition to being president of ABC Sports. That same year, ABC launched a major expansion of its office facilities in New York City. The company first constructed a new 10-story building on land previously occupied by an abandoned warehouse on the corner of Columbus Avenue and West 66th Street; the facility that was built in its place is nicknamed "7 Lincoln Square" (although it is actually located at 149 Columbus Avenue). Meanwhile, a former parking lot, located at 30 West 67th Street, was transformed into an impressive 15-story building. Both buildings were completed in June 1979. WABC-TV moved its operations from offices at 77 West 66th Street to 149 Columbus Avenue, freeing up space for the ABC network to house some of its operations.	
Case 4	<i>CMC partial gain (Baseline~ / CMC~)</i>
Question: Where did the residents of Antioch flee to?	
Gold answer: The city's residents fled to the north	
Baseline: the north	EM = 0 / F1 = 0.40
CMC: to the north	EM = 0 / F1 = 0.55
<i>Context:</i> The plague struck various countries in the Middle East during the pandemic, leading to serious depopulation and permanent change in both economic and social structures. As it spread to western Europe, the disease entered the region from southern Russia also. By autumn 1347, the plague reached Alexandria in Egypt, probably through the port's trade with Constantinople, and ports on the Black Sea. During 1347, the disease travelled eastward to Gaza, and north along the eastern coast to cities in Lebanon, Syria and Palestine, including Ashkelon, Acre, Jerusalem, Sidon, Damascus, Homs, and Aleppo. In 1348–49, the disease reached Antioch. The city's residents fled to the north, most of them dying during the journey, but the infection had been spread to the people of Asia Minor.	

Table 18: Qualitative case study on SQuAD validation set (full-data setting, GPT2-Large, $r = 4$, Llama). *Baseline* receives the local context window without compression. Gold answers are shown in blue, correct predictions in green, and incorrect predictions in red. EM and F1 are per-example scores.